Ilyas BELAÏD
Moilim ABDALLAH
Nassim BOUAZZA
ING1 LSI APP

PROJET FINAL

CAMPUS EVENT

Concevoir et réaliser un système d'information de gestion d'événements universitaires

Module : Introduction aux Systèmes d'information

Ce projet est une mise en application autonome des notions étudiées pendant le module. Contrairement aux TD et TP, les solutions ne sont pas données étape par étape. Vous devez réutiliser les méthodes vues avec GESTFORM et les adapter au nouveau contexte CAMPUS EVENT.

1. Contexte du projet

Une université organise régulièrement des conférences, ateliers, forums entreprises, événements associatifs, journées portes ouvertes et rencontres professionnelles. Aujourd'hui, la gestion de ces événements et des inscriptions repose sur plusieurs fichiers et formulaires indépendants.

Cette organisation entraîne plusieurs difficultés :

- des informations dispersées et parfois incohérentes ;
- des risques de double inscription ;
- une difficulté à connaître le nombre de places restantes ;
- un suivi incomplet des participants ;
- une vision limitée pour les organisateurs et l'administration.

Objectif métier: L'université souhaite centraliser les événements, les étudiants et les inscriptions dans un même système nommé CAMPUS EVENT.

2. Scénario principal

Un étudiant consulte la liste des événements disponibles, sélectionne un événement et demande à s'y inscrire. Le système doit vérifier les informations, l'existence de l'étudiant et de l'événement, la disponibilité des places et l'absence de double inscription. Si les conditions sont satisfaites, une inscription est créée.

L'inscription possède initialement le statut :

pending → accepted / rejected

Extension possible : Si l'événement est complet, vous pourrez proposer une gestion de liste d'attente avec un statut waitlisted ou un modèle dédié.

3. Partie 1 — Analyser le système d'information

Avant de penser au code, analysez CAMPUS EVENT comme un système d'information. Votre analyse doit partir du métier

avant de lister les technologies.

3.1 Identifier les acteurs

Identifiez les acteurs du système et précisez leur rôle.

Acteur	Rôle	Informations consultées	Informations créées / modifiées
Étudiants	Consulte les événements et s'inscrit	Liste et détail des événements, places restantes, ses inscriptions	Demande d'inscription (student_id, event_id)
Organisateur	Crée et pilote ses événements, valide les inscriptions	Inscriptions de ses événements, taux de remplissage	Événements (titre, date, lieu, capacité), statut des inscriptions (accepted / rejected)
Administration	Supervise l'ensemble, gère les étudiants	Tous les événements, étudiants, inscriptions, statistiques	Fiches étudiants, paramètres globaux, arbitrage des litiges
Système CAMPUS EVENT	Applique les règles métier	Étudiants, événements, inscriptions	Inscriptions (statut initial pending), calcul des places restantes

3.2 Identifier les processus métier

Identifiez au minimum trois processus métier et décomposez-les en quelques étapes.

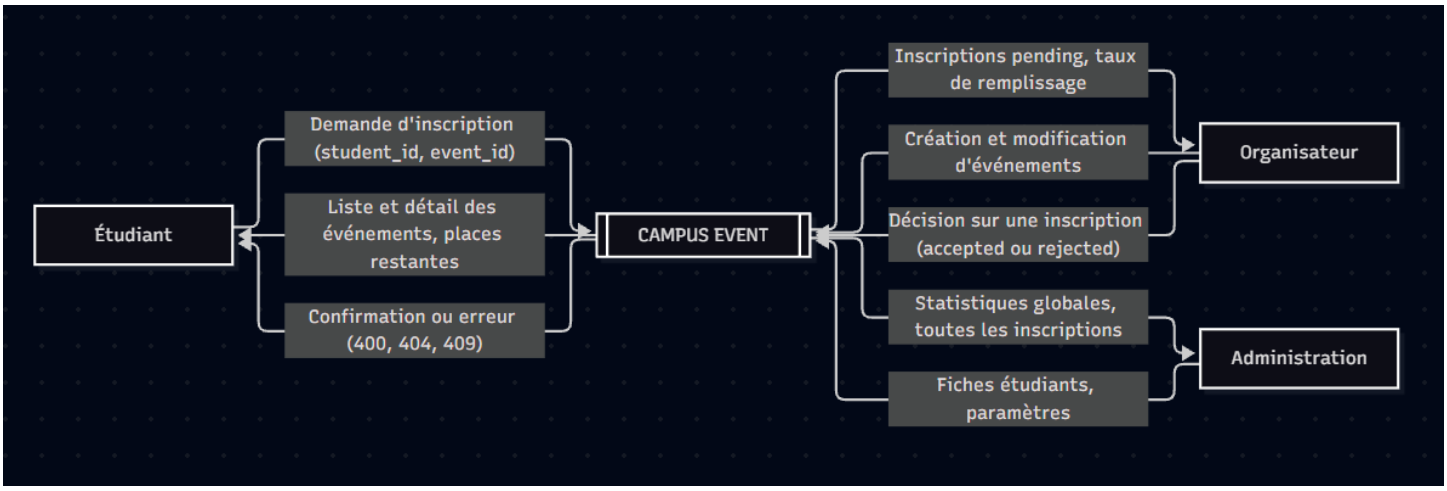
Processus	Objectif métier	Étapes principales	Acteur(s) responsable(s)
Inscription à un événement	Inscrire un étudiant sans doublon ni dépassement de capacité	1. Consulter les événements 2. Choisir un événement 3. Envoyer la demande 4. Vérifier étudiant, événement, places, doublon 5. Créer l'inscription 6. Confirmer à l'étudiant	Étudiant, Système
Validation des inscriptions	Contrôler les participants admis	1. Consulter les inscriptions pending 2. Examiner la demande 3. Passer le statut à accepted ou rejected 4. Mettre à jour les places	Organisateur, Système
Gestion des événements	Maintenir un catalogue à jour	1. Créer l'événement 2. Fixer date, lieu, capacité 3. Publier 4. Modifier ou clôturer	Organisateur, Administration
Pilotage et statistiques	Donner une vision globale	1. Agréger les inscriptions 2. Calculer remplissage et participation 3. Consulter les tableaux de bord	Administration, Organisateur

3.3 Identifier les informations

Catégorie	Informations principales	Qui les crée ?	Qui les consulte ?
Étudiants	id, nom, prénom, email, filière, année	Administration	Administration, Système, Organisateur
Événements	id, titre, date, lieu, capacité, places disponibles	Organisateur	Étudiant, Organisateur, Administration
Inscriptions	id, student_id, event_id, date d'inscription, statut (pending / accepted / rejected)	Étudiant (demande), Système (création) ; statut modifié par l'Organisateur	Étudiant, Organisateur, Administration
Pilotage / statistiques	Nombre d'inscrits par événement, taux de remplissage, répartition des statuts	Système (calcul automatique)	Administration, Organisateur

3.4 Représenter les flux d'information

Représentez les principaux flux d'information entre les acteurs et CAMPUS EVENT. Votre schéma doit répondre à : qui envoie quoi à qui ?



4. Partie 2 — Proposer l'architecture du système

Proposez une architecture 3-tiers en distinguant présentation, logique métier et données.

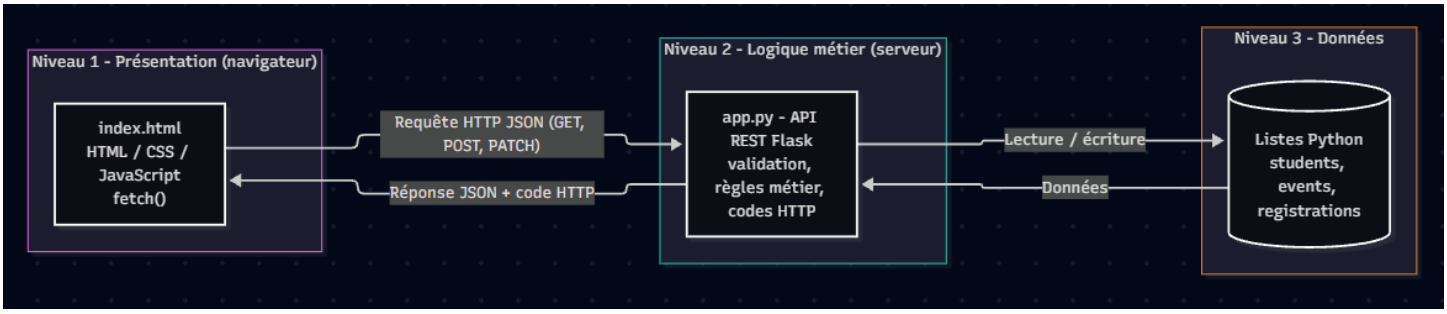
Niveau	Rôle	Traitements / responsabilités	Technologie proposée
Présentation	Interface utilisateur	Afficher les événements, envoyer les requêtes avec fetch(), afficher les messages et erreurs	HTML / CSS / JavaScript
Logique métier	API REST	Valider les champs, vérifier existence, places et doublons, créer et modifier les inscriptions, choisir les codes HTTP	Python / Flask
Données	Stockage	Conserver étudiants, événements, inscriptions	Listes Python (SQLite en bonus)

Technologies autorisées :

- Présentation : HTML / CSS / JavaScript.
- Logique métier : Python / Flask.
- Données : listes Python. SQLite peut être utilisé en bonus.

4.1 Schéma d'architecture

Dessinez votre architecture et montrez clairement les échanges entre les trois niveaux. Le frontend ne doit pas accéder directement aux données.



5. Partie 3 — Concevoir l'API REST

Identifiez les ressources REST du système. Votre API devra au minimum gérer les étudiants, les événements et les inscriptions.

5.1 Ressources

Ressource métier	Nom REST proposé	Exemple de données importantes
Étudiant	students	id, nom, prenom, email, filiere, annee
Événement	events	id, titre, date, lieu, capacite, places_disponibles
Inscription	registrations	id, student_id, event_id, date_inscription, status (pending / accepted / rejected)
Autre ressource éventuelle	statistics (non implémentée dans app.py)	nombre d'inscrits par événement, taux de remplissage, répartition des statuts

5.2 Endpoints

Complétez le tableau à partir des besoins métier.

Besoin métier	Méthode HTTP	Endpoint	Code succès attendu
Consulter tous les événements	GET	/events	200
Consulter un événement précis	GET	/events/<id>	200
Consulter les étudiants	GET	/students	200
Créer une inscription	POST	/registrations	201
Consulter les inscriptions	GET	/registrations	200
Consulter une inscription	GET	/registrations/<id>	200
Modifier le statut d'une inscription	PATCH	/registrations/<id>	200

Règle REST: L'URL identifie la ressource ; la méthode HTTP indique l'action réalisée sur cette ressource.

6. Partie 4 — Développer l'API avec Flask

Créez un fichier principal nommé :

app.py

Votre application utilisera initialement des listes Python.

Structure minimale
students = [...] events = [...] registrations = []

6.1 Endpoints obligatoires

- GET /events
- GET /events/<id>
- GET /students
- POST /registrations
- GET /registrations
- GET /registrations/<id>
- PATCH /registrations/<id>

6.2 Création d'une inscription

Le client enverra un JSON contenant au minimum :

Exemple de corps JSON

```
{
  "student_id": 1,
  "event_id": 2
}
```

6.3 Règles métier obligatoires

Règle	Comportement attendu	Code HTTP attendu
Champs obligatoires absents	Refuser la demande (message d'erreur JSON)	400
Étudiant inexistant	Refuser la demande	404
Événement inexistant	Refuser la demande	404
Événement complet	Refuser la demande (pas de liste d'attente)	409
Double inscription	Refuser la demande	409
Création réussie	Créer l'inscription avec le statut pending et retirer une place	201
Modification du statut réussie	Passer à accepted ou rejected (si rejected, la place est libérée)	200

7. Partie 5 — Tester l'API avec Postman

Construisez une campagne de tests. Chaque test doit préciser la requête, les données envoyées si nécessaire, le résultat attendu et le résultat réellement obtenu.

Test	Résultat attendu	Résultat obtenu	Capture / commentaire
GET /events	200	200, liste de 3 événements	Capture 1 : les 3 événements avec capacité et places
GET événement existant	200	200, événement 1 « Conférence IA »	Capture 2
GET événement inexistant	404	404, {"error": "Evenement introuvable"}	Capture 3
POST inscription valide	201	201, inscription créée, statut pending	Capture 4 : l'événement 3 passe à 0 place
POST JSON incomplet	400	400, {"error": "student_id et event_id sont obligatoires"}	Capture 5 : event_id absent
POST étudiant inexistant	404	404, {"error": "Etudiant introuvable"}	Capture 6 : student_id 99
POST événement inexistant	404	404, {"error": "Evenement introuvable"}	Capture 7 : event_id 99
POST événement complet	409	409, {"error": "Evenement complet"}	Capture 8 : événement 3 plein depuis le test 4
POST double inscription	409	409, {"error": "Etudiant deja inscrit a cet evenement"}	Capture 9 : second envoi du même corps
PATCH statut	200	200, statut passé à accepted	Capture 10

Test 1 :

Body 200 e.g.

{} JSON


```
[
    {
        "capacite": 100,
        "date": "2026-12-03",
        "id": 1,
        "lieu": "Amphi A",
        "places_disponibles": 100,
        "titre": "Conférence IA"
    },
    {
        "capacite": 50,
        "date": "2026-12-10",
        "id": 2,
        "lieu": "Hall principal",
        "places_disponibles": 50,
        "titre": "Forum entreprises"
    },
    {
```


Test 2

Body 200 e.g.

{} JSON


```
{
    "capacite": 100,
    "date": "2026-12-03",
    "id": 1,
    "lieu": "Amphi A",
    "places_disponibles": 100,
    "titre": "Conférence IA"
}
```


Test 3

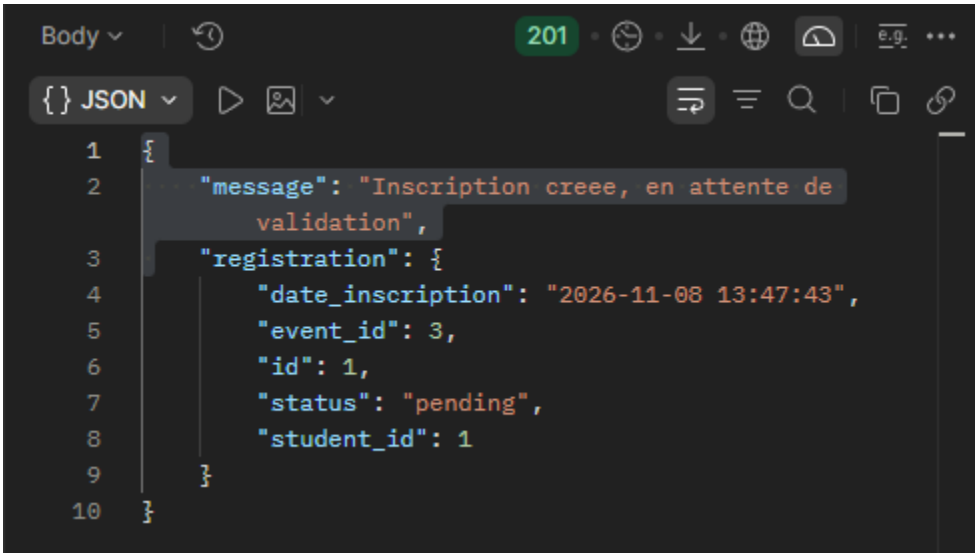
Body 404 e.g.

Investigate 404 error Diagnose Decline Think Harder

{} JSON


```
{
    "error": "Evenement introuvable"
}
```

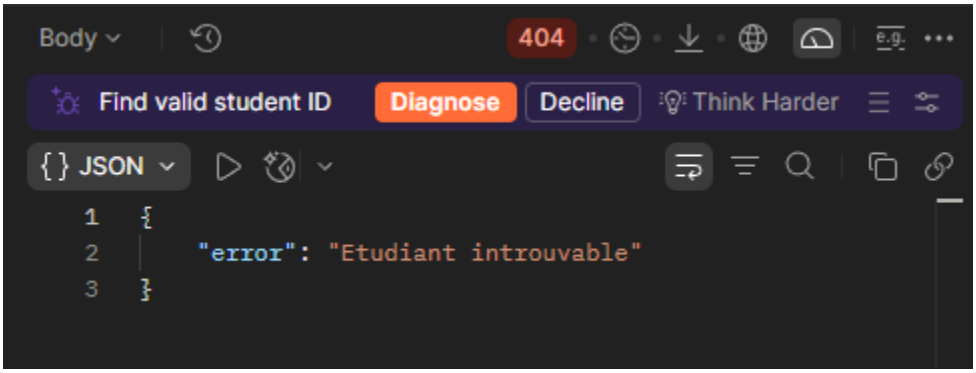

Test 4



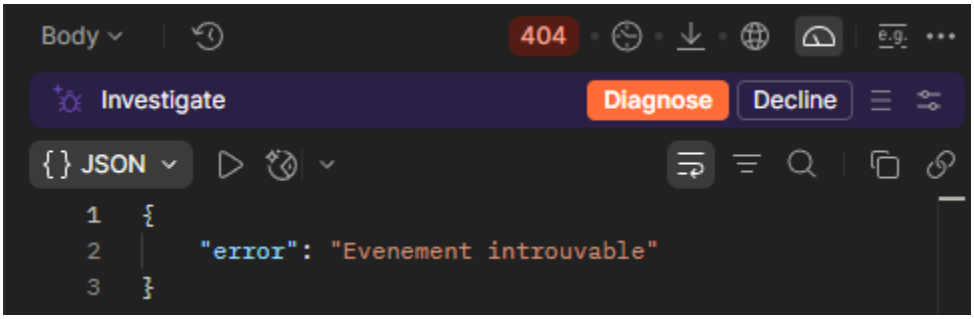
Test 5



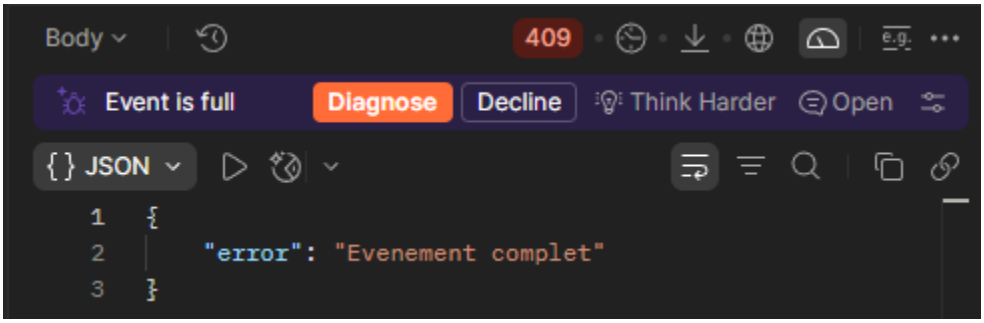
Test 6



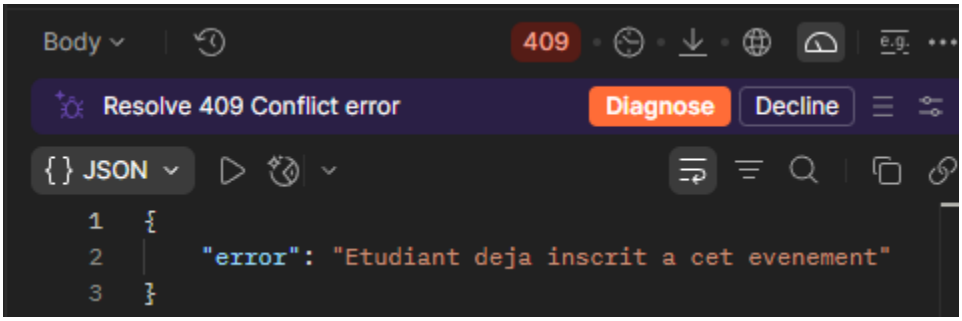
Test 7



Test 8



Test 9



Test 10



8. Partie 6 — Modélisation UML

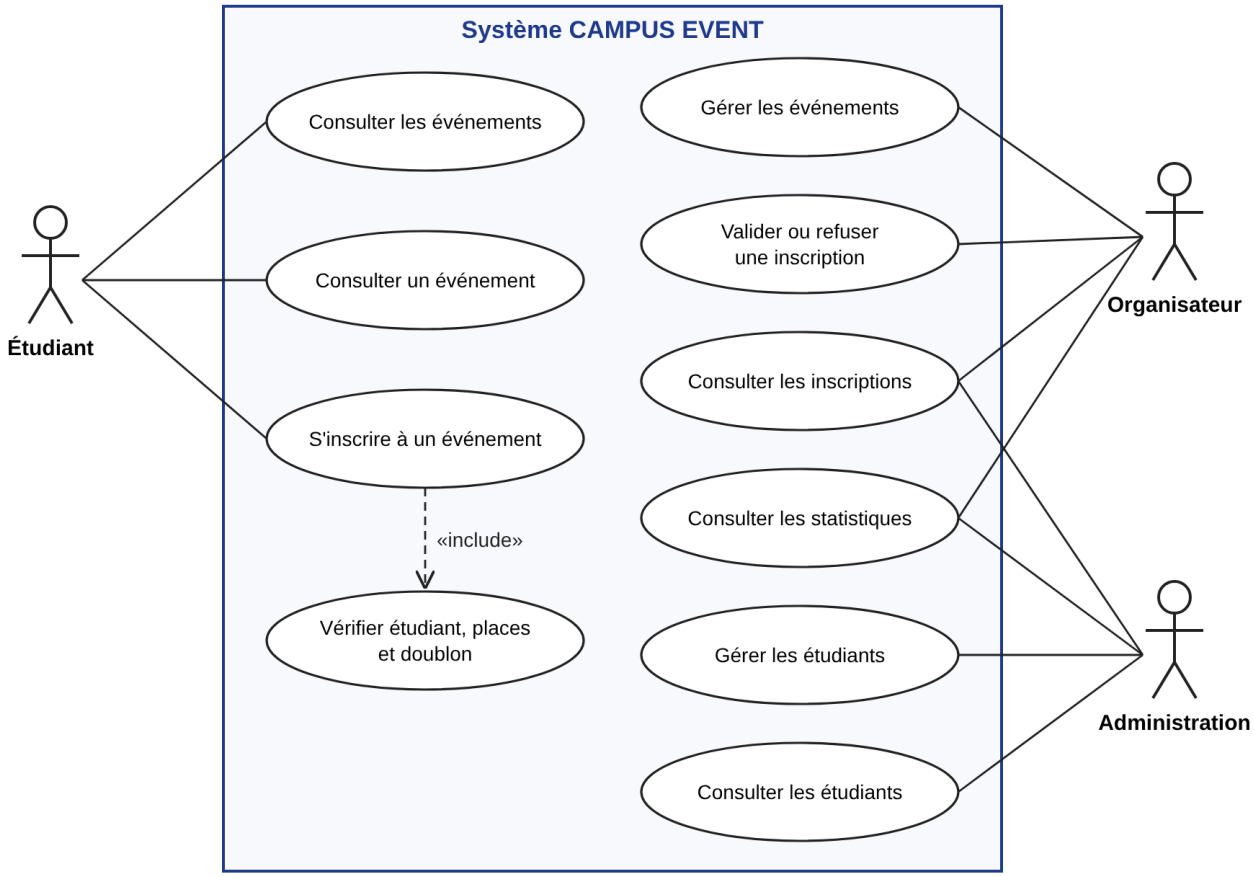
Vous devez modéliser CAMPUS EVENT avec les quatre diagrammes étudiés en cours. Chaque diagramme répond à une question différente.

Diagramme	Question principale
Cas d'utilisation	Qui utilise le système et pour faire quoi ?
Activité	Comment se déroule le processus ?
Séquence	Qui communique avec qui, dans quel ordre ?
Classes	Quelles sont les entités et leurs relations ?

Les quatre diagrammes UML doivent représenter le même système. Les acteurs, processus, échanges et entités identifiés dans les différentes parties du projet doivent rester cohérents d'un diagramme à l'autre.

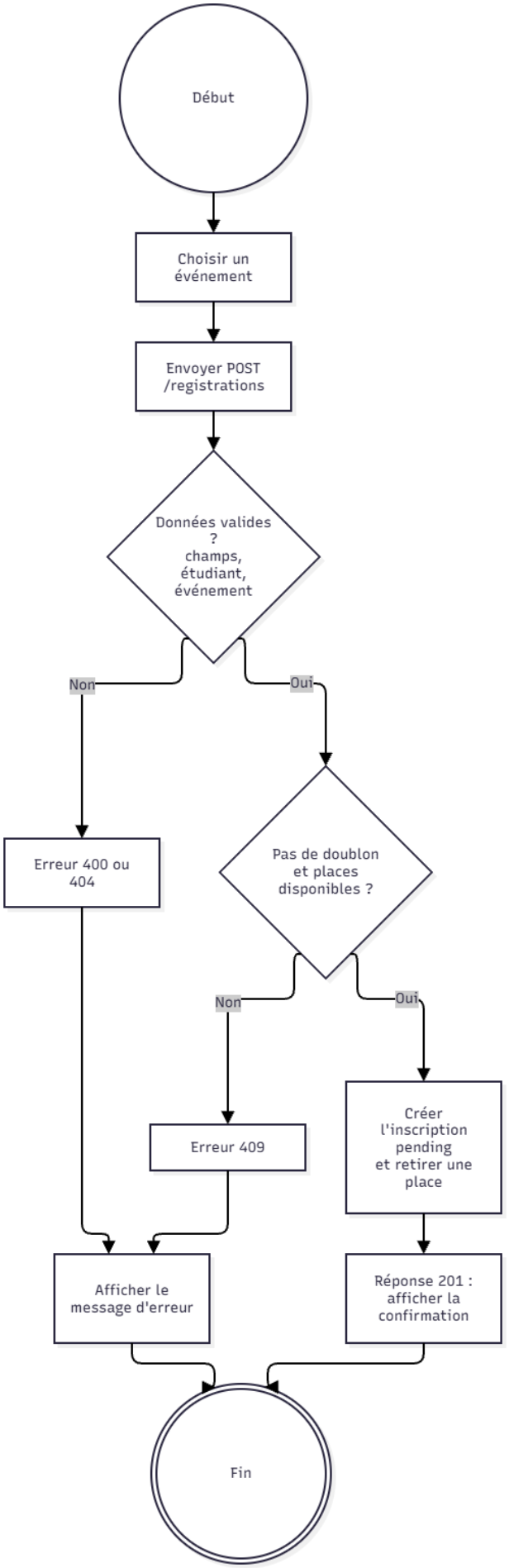
8.1 Diagramme de cas d'utilisation

Identifiez les acteurs et les principaux cas d'utilisation de CAMPUS EVENT. Le diagramme doit montrer clairement la frontière du système.



8.2 Diagramme d'activité — « S'inscrire à un événement »

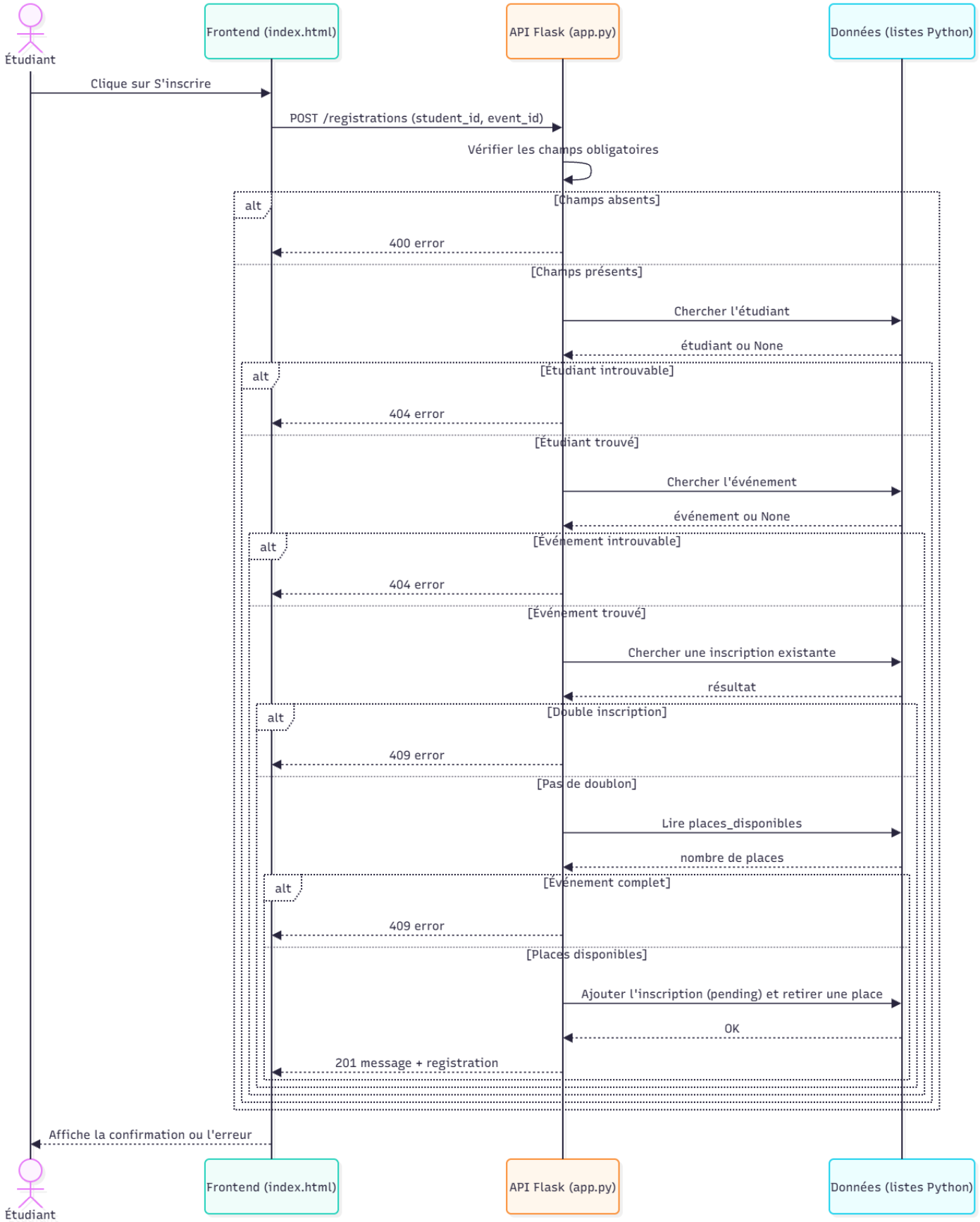
Le diagramme doit montrer le début, les actions, les vérifications, au moins une décision, les chemins succès / échec et la fin du processus.



8.3 Diagramme de séquence — inscription à un événement

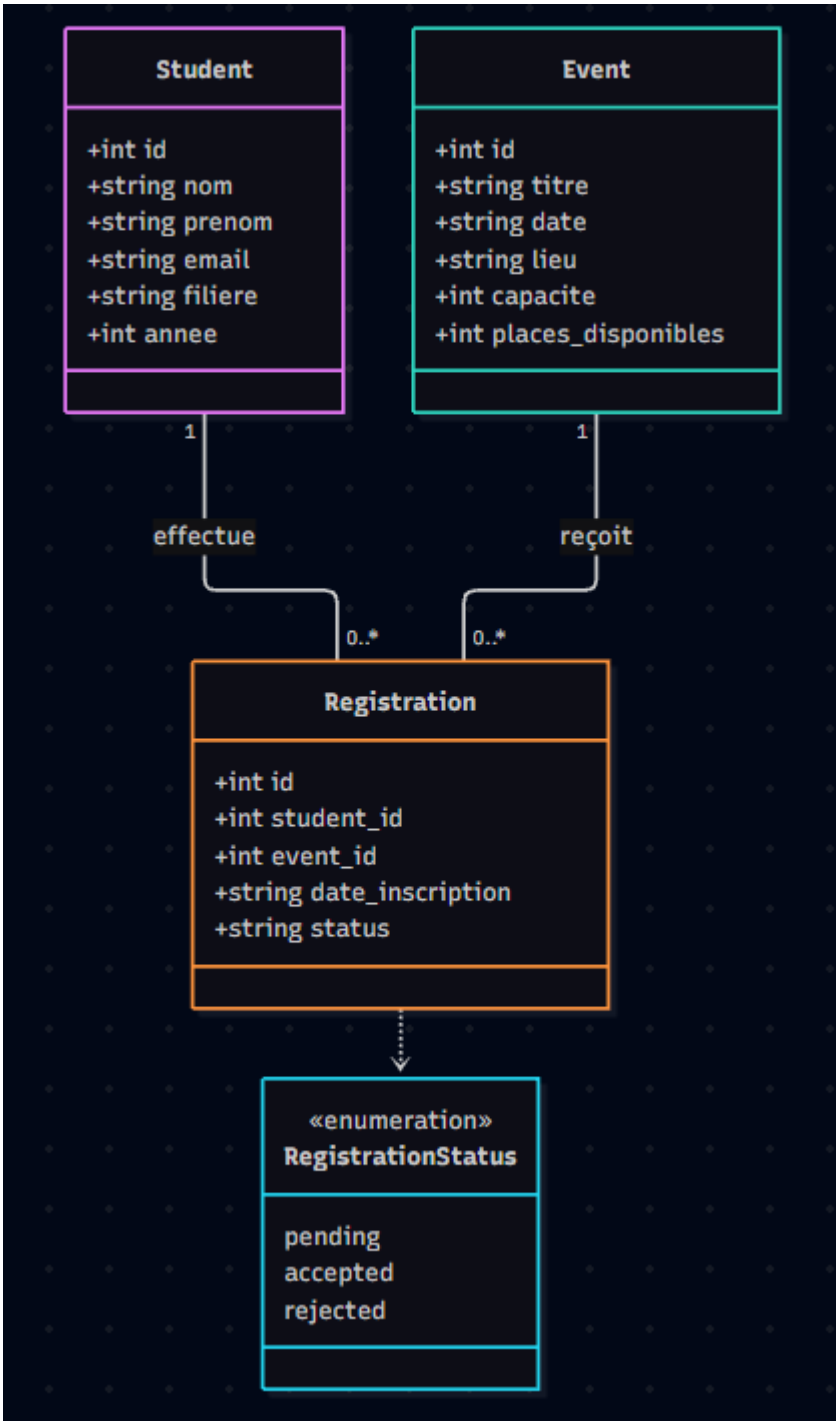
Votre diagramme doit faire apparaître au minimum :

- l'Étudiant ;
- le Frontend ;
- l'API / Backend ;
- les Données ;
- la requête POST /registrations ;
- les vérifications métier ;
- la réponse de l'API ;
- au moins un scénario alternatif.



8.4 Diagramme de classes

Le modèle doit contenir au minimum Student, Event et Registration, avec attributs, associations et cardinalités.



9. Partie 7 — Frontend avec Fetch

Créez une page index.html permettant de communiquer avec votre API Flask.

Travail obligatoire

Votre interface doit au minimum :

- appeler l'endpoint GET /events avec fetch() ;

- récupérer la réponse JSON retournée par l'API ;
- afficher dynamiquement la liste des événements ;
- afficher pour chaque événement au minimum :
    - son titre ;
    - sa capacité ;
    - le nombre de places disponibles ;
- afficher un message clair si l'API ne répond pas ou si une erreur se produit.

L'objectif n'est pas de réaliser une interface graphique complexe, mais de montrer qu'un frontend peut consommer les données fournies par une API REST.

Fonctionnalités bonus

Les fonctionnalités suivantes sont facultatives :

- ajouter un formulaire d'inscription ;
- envoyer une requête POST /registrations avec fetch() ;
- afficher le message de confirmation retourné par l'API ;
- afficher les erreurs métier retournées par l'API (400, 404, 409, etc.) ;
- améliorer la présentation avec CSS.

Résultat minimum attendu

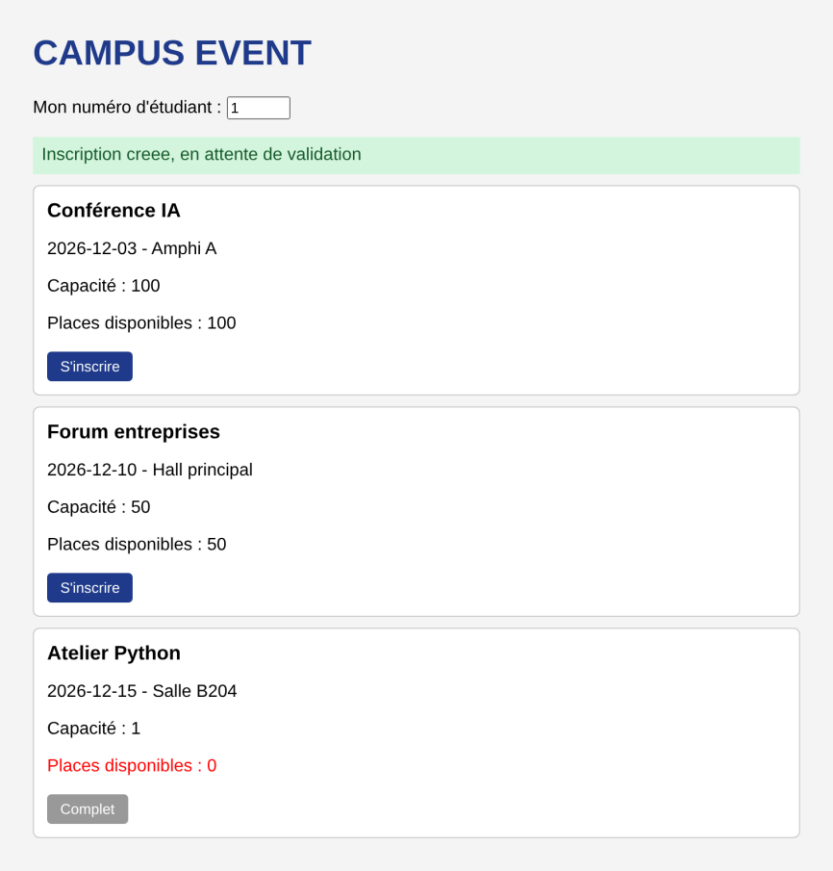
Lorsque l'API Flask fonctionne et que l'utilisateur ouvre index.html, la page doit afficher les événements obtenus depuis : **GET /events**

Les données affichées ne doivent donc pas être écrites directement dans la page HTML : elles doivent provenir de l'API.

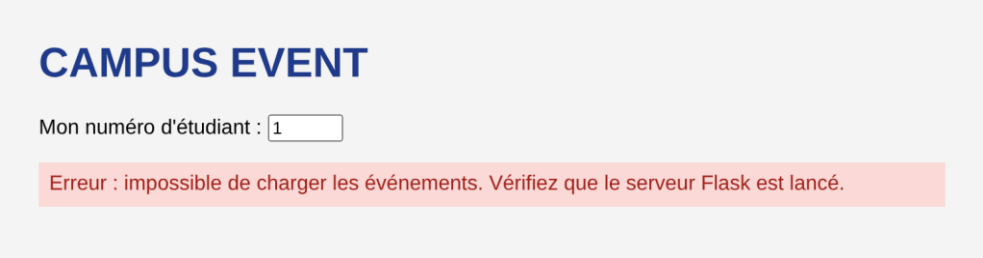
Réalisation

Au chargement, index.html appelle GET /events avec fetch(). La page lit la réponse JSON et crée une carte par événement avec son titre, sa date, son lieu, sa capacité et ses places disponibles. Aucune donnée n'est écrite en dur dans la page.

Nous avons aussi réalisé les fonctionnalités bonus. L'étudiant saisit son numéro puis clique sur S'inscrire. La page envoie POST /registrations avec fetch(). Elle affiche le message de l'API en vert si l'inscription est créée, ou l'erreur et son code HTTP en rouge. Quand un événement n'a plus de place, le bouton devient gris et affiche Complet.



Capture 11. Inscription de l'étudiant 1 à l'Atelier Python. La dernière place est prise et l'événement passe à Complet.



Capture 12. Message affiché quand le serveur Flask est arrêté.

10. Livrables

Vous devez remettre un dossier structuré de la manière suivante :

Arborescence attendue
CAMPUS_EVENT/ ├── app.py ├── index.html ├── README.md └── rapport.pdf

10.1 Contenu du rapport

- présentation du système et du besoin ;
- analyse du SI : acteurs, processus, informations et flux ;
- architecture proposée ;
- ressources REST et tableau des endpoints ;
- règles métier principales ;
- campagne de tests Postman avec captures ;

- diagramme de cas d'utilisation ;
- diagramme d'activité ;
- diagramme de séquence ;
- diagramme de classes ;
- difficultés rencontrées et choix réalisés ;

- conclusion.

10.2 README

- installation de Flask ;
- commande pour lancer l'application ;
- adresse du serveur ;
- liste des endpoints ;
- exemples de requêtes ;
- fonctionnalités bonus éventuelles.

11. Barème indicatif /20

Critère	Points
Analyse du système d'information	2
Architecture proposée	2
Conception REST / endpoints	2
API Flask fonctionnelle	4
Règles métier et codes HTTP	2
Tests Postman	1
Diagramme de cas d'utilisation	1,5
Diagramme d'activité	1,5
Diagramme de séquence	1,5
Diagramme de classes et cardinalités	1,5
Qualité du code et documentation	1
TOTAL	20

12. Difficultés rencontrées et choix réalisés

Choix réalisés

Stockage en listes Python. C'est la solution demandée par le sujet. Les données reviennent à leur état initial à chaque redémarrage du serveur. Cela permet aussi de rejouer la campagne Postman depuis un état connu.

Place réservée dès la demande. Une inscription pending retire une place. Ainsi, un événement ne peut jamais être dépassé pendant que des demandes attendent d'être validées. Si l'inscription passe à rejected, la place est rendue.

Double inscription. Un étudiant ne peut pas avoir deux inscriptions actives au même événement. Une inscription refusée ne bloque pas une nouvelle demande.

Inscription déjà traitée. Une inscription accepted ou rejected ne peut plus être modifiée et l'API renvoie 409. On évite ainsi de rendre deux fois la même place.

Contrôle des identifiants. student_id et event_id doivent être des entiers positifs. Un corps qui n'est pas un objet JSON, un identifiant en texte ou un booléen sont refusés avec le code 400.

Événement de test à une place. L'Atelier Python a une capacité de 1 pour tester facilement le cas de l'événement

complet.

CORS. Les en-têtes CORS sont ajoutés directement dans app.py, sans bibliothèque supplémentaire, pour que index.html puisse appeler l'API.

Difficultés rencontrées

Blocage CORS. Au début, le navigateur bloquait les appels fetch() de index.html vers l'API. Il a fallu ajouter les en-têtes CORS côté Flask.

Choix des codes HTTP. Il a fallu bien distinguer 400 pour une requête mal formée, 404 pour une ressource qui n'existe pas et 409 pour un conflit avec une règle métier.

Travail à trois. L'analyse, le code, les diagrammes et les tests ont été faits en parallèle. Il a fallu vérifier à la fin que les acteurs, les entités et les champs restaient les mêmes partout.

13. Conclusion

CAMPUS EVENT centralise les étudiants, les événements et les inscriptions dans un seul système organisé en trois niveaux. L'API Flask applique toutes les règles métier demandées. Elle contrôle les champs obligatoires, l'existence de l'étudiant et de l'événement, les places disponibles et la double inscription. La campagne Postman montre qu'elle renvoie les bons codes HTTP. Le frontend affiche les événements avec fetch() et permet de s'inscrire.

Le projet pourrait être amélioré avec une base SQLite pour conserver les données, une liste d'attente avec le statut waitlisted et une ressource statistics pour les organisateurs.

Annexe — Détail des requêtes des tests Postman

Requêtes et données envoyées pour chacun des tests de la partie 7, dans l'ordre d'exécution, sur un serveur fraîchement démarré. Le test 8 utilise un autre étudiant que le test 4 : l'événement 3 (capacité 1) est alors complet, et la vérification de double inscription ne s'applique pas.

N°	Test	Requête	Corps JSON envoyé (Content-Type: application/json)
1	GET /events	`GET /events`	`(aucun)`
2	GET événement existant	`GET /events/1`	`(aucun)`
3	GET événement inexistant	`GET /events/99`	`(aucun)`
4	POST inscription valide	`POST /registrations`	`{"student_id": 1, "event_id": 3}`
5	POST JSON incomplet	`POST /registrations`	`{"student_id": 1}`
6	POST étudiant inexistant	`POST /registrations`	`{"student_id": 99, "event_id": 1}`
7	POST événement inexistant	`POST /registrations`	`{"student_id": 1, "event_id": 99}`
8	POST événement complet	`POST /registrations`	`{"student_id": 2, "event_id": 3}`
9	POST double inscription	`POST /registrations`	`{"student_id": 1, "event_id": 3}`
10	PATCH statut	`PATCH /registrations/1`	`{"status": "accepted"}`